

## Getting started with Amazon Bedrock AgentCore



[Getting started with Amazon Bedrock AgentCore](#) > Lab 4: Deploy the Agent to production with Observability

# Lab 4: Deploy the Agent to production with Observability

## Overview

In prior labs, we made from our Customer Support Agent a tool-powered memory-aware intelligent assistant for the end users. But how to move it from a laptop based prototype to a scalable production endpoint all while securely granting our users access to it?

[AgentCore Runtime](#) is a secure, serverless runtime designed for deploying and scaling AI agents and tools. It supports any frameworks, models, and protocols, enabling developers to transform local prototypes into production-ready solutions with minimal code changes. This will transform our prototype into a production-ready system that can handle real-world traffic with full monitoring and automatic scaling. Additionally, AgentCore Runtime integrates seamlessly with [AgentCore Observability](#) to provide full visibility into your agent's behavior in production.

### ⚠️ Prerequisite for AgentCore Observability

To view metrics, spans, and traces generated by the AgentCore Runtime using AgentCore Observability, you first need to complete a one-time setup to turn on Amazon CloudWatch Transaction Search. More specifically, [following this documentation](#), you have to:

1. Open the CloudWatch console at <https://console.aws.amazon.com/cloudwatch/>.
2. From the navigation pane, under **Application Signals**, choose **Transaction Search**.
3. Choose **Enable Transaction Search**.
4. Select the box to ingest spans as structured logs, and enter a percentage of spans to be indexed. You can index spans at 1% for free and change the percentage later based on your requirements.

Key benefits:

- **Framework Agnostic:** Works with any Python-based agentic framework (Strands Agents, LangGraph, CrewAI)
- **Model Flexible:** Support for LLMs in Amazon Bedrock, OpenAI, and other LLM providers
- **Production Ready:** Built-in health checks and monitoring
- **Easy Integration:** Easy management through AWS console and APIs and support for real-world production workloads.
- **Scalable:** Designed for enterprise workloads

## What we are building

In this lab, by means of the Amazon BedrockAgentCore Python SDK, you will wrap your AgentCore Memory-enhanced (Lab 2) Strands Agent (Lab 1) with a `BedrockAgentCoreApp` and define a dedicated `invoke` entrypoint. Then, you will use the `configure` and `launch` capabilities of the SDK to deploy your agent to AgentCore Runtime. Your application is then able to invoke this agent via the same SDK.

By the end of this lab, you'll understand:

- How to initialize the AgentCore Runtime App and define the entrypoint for runtime invocation
- How to configure, build and deploy your AgentCore Runtime instance
- How to programmatically invoke your agent from within the AgentCore Runtime
- How to monitor and analyze your agent's traces with AgentCore Observability

## Architecture diagram

### ▼ Prerequisites

At an AWS Event (Setup)

Self paced (Setup)

Sagemaker AI Studio

Amazon Bedrock AgentCore Fundamentals (Optional)

Lab 1: Create the Agent Prototype

Lab 2: Enhance your Agent with Memory

Lab 3: Scale with Gateway and Identity

**Lab 4: Deploy the Agent to production with Observability**

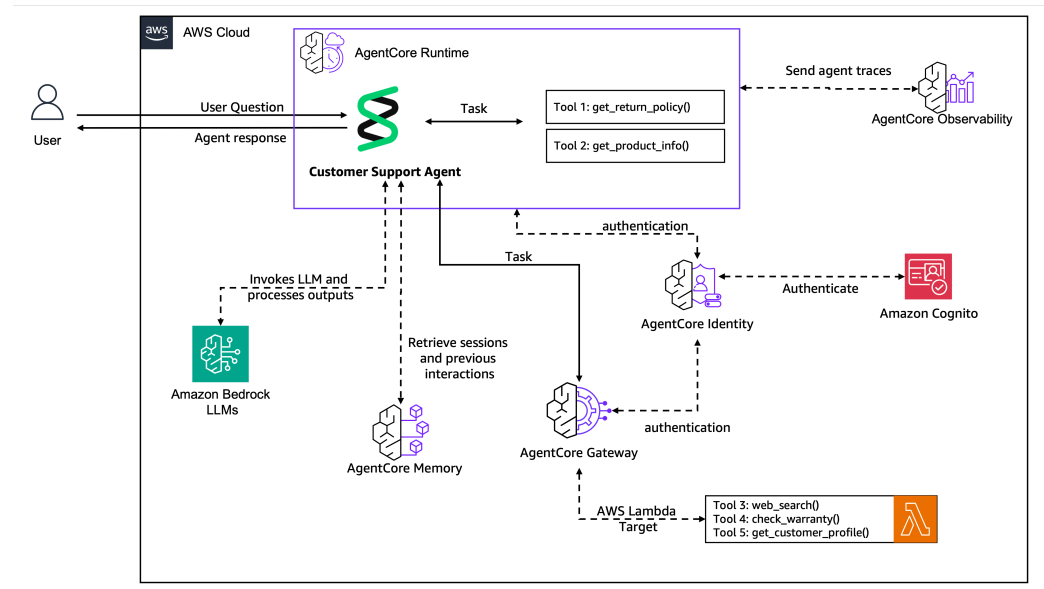
(Optional) Lab 5: Build a Customer-Facing Frontend Application

Lab 6: Clean up

[AgentCore](#)

[AgentCore Documentation](#)

Below is the updated Solution Architecture diagram including the Agent deployment within the AgentCore Runtime:



**Incremental Workshop Journey:** Lab 4 of 5-part progressive workshop.

## Steps to follow

**Note:** The code snippet in the following steps are shown for reference purposes only. The complete implementation is provided in the accompanying Jupyter notebook for this lab section.

### Step 1: Prepare Your Agent for AgentCore Runtime

Let's first define the necessary AgentCore Runtime components via Python SDK within our previous local agent implementation. Expand the following snippet to see the additional required commands:

► **Code Snippet**

### Step 2: Deploy to AgentCore Runtime

[AgentCore Starter Toolkit](#) allows us to configure, build and deploy our `BedrockAgentCoreApp` to the AgentCore Runtime.

► **Code Snippet**

**Note:** You might get a warning mentioning "Platform Mismatch". It's only for information, and safe to ignore for the scope of this workshop.

### Step 3: Invoke Your Deployed Agent

Now that our agent is deployed and ready, we can invoke it with the right Cognito authorization token using the same AgentCore Starter Toolkit:

► **Code Snippet**

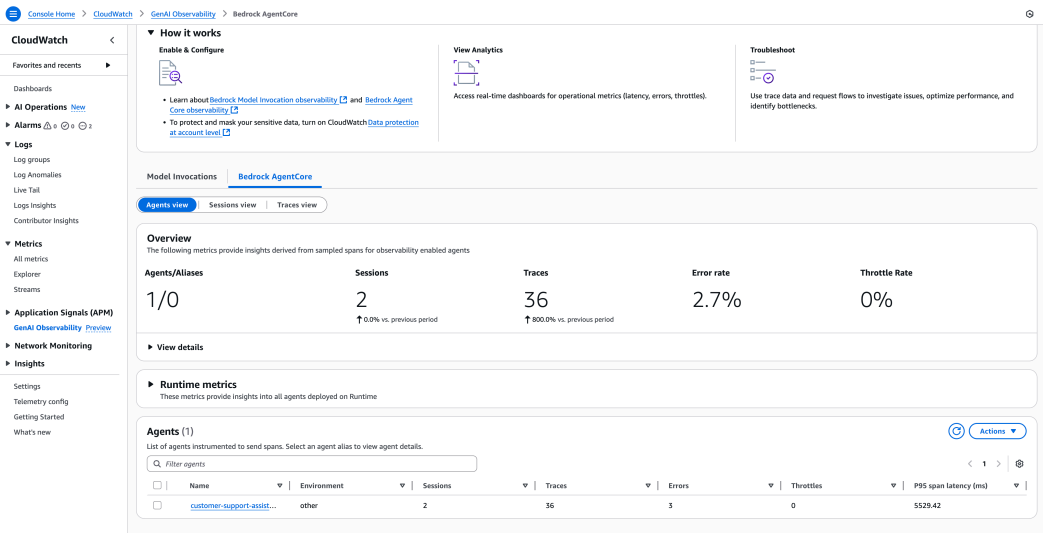
And it is all it takes to have a secure and scalable endpoint for our Agent with no need to manage all the underlying infrastructure!

## Step 4: AgentCore Observability

[AgentCore Observability](#) provides monitoring and tracing capabilities for AI agents using Amazon OpenTelemetry Python Instrumentation and Amazon CloudWatch GenAI Observability.

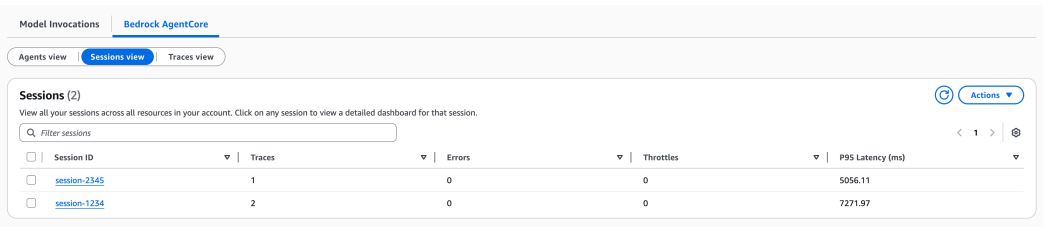
## Agents

Default AgentCore Runtime configuration allows for logging our agent's traces on CloudWatch by means of **AgentCore Observability**. These traces can be seen on the AWS CloudWatch GenAI Observability dashboard. Navigate to Cloudwatch → GenAI Observability → Bedrock AgentCore.



## Sessions

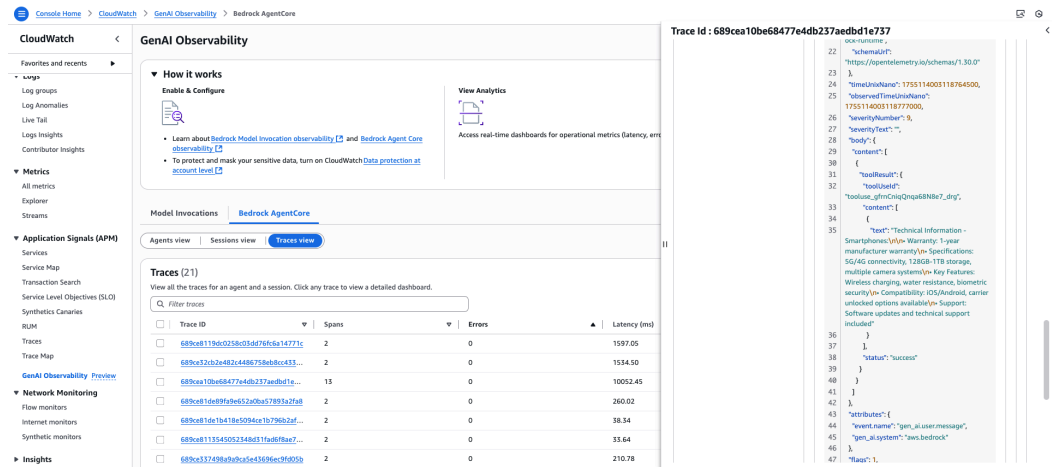
The Sessions view shows the list of all the sessions associated with all agents in your account.



## Traces

Trace view lists all traces from your agents in this account. To work with traces:

- Choose Filter traces to search for specific traces.
- Sort by column name to organize results.
- Under Actions, select Logs Insights to refine your search by querying across your log and span data or select Export selected traces to export.



**Congratulations - you successfully completed Lab 4: Hosting and monitoring agents securely and at scale with AgentCore Runtime and Observability!**

You've configured, built and deployed your customer support agent to the AgentCore Runtime for further invocations. Now you have:

- a deployed production-ready agent, that the end users can access securely
- a serverless solution which will scale automatically to meet the ever-changing demand
- a built-in monitoring and traceability of all of the agent's steps

**Resources**

- [Amazon Bedrock AgentCore Runtime Documentation](#)
- [AgentCore Runtime Code Samples](#)
- [Amazon Bedrock AgentCore Observability Documentation](#)
- [AgentCore Observability Code Samples](#)

**Try it out**

**At an AWS event**

If you are following the workshop via workshop studio, now go to JupyterLab in SageMaker Studio. In the JupyterLab UI navigate to `lab-04-agentcore-runtime.ipynb`

**Self paced**

For the complete working implementation and examples: [Deploy to Production - Use AgentCore Runtime with Observability](#).

**Ready to Continue?** → Go to [Lab 5: Build User Interface - Create a customer-facing application](#)